Assignment 6

Altering and Updating Tables, and Triggers

The premise of this homework is to provide you with practice editing and “dummy-proofing” existing databases.

In this assignment you will be given an updated SQL file containing the same tables WITHOUT any constraints added. (Assignment 5 had constraints implemented for you). This assignment will also provide you with another insert file with BAD input (this is intentional).

Hypothetically in this assignment, you are working for a company that has a pre-established database. The original database designers have not taken a proper class on database design and management, so they have left some vulnerabilities towards bad input.

Part 1 - Cleaning up the database
Goal: Create a savepoint and delete bad data entries

There are many ways when dealing with bad data. It can come in the form of missing data, misinputs, or simply values that should not be accepted in the context of the database. We will not go over all the different avenues to accommodate these instances. In this assignment, we will simply choose the trivial route and delete the “BAD” INPUTS.

We could simply DELETE entries, but that would be bad practice, as it will be rather difficult to recover them if we accidentally deleted the wrong row. Instead we will utilize savepoints to have a method to rollback to a previous state, in case we make a mistake.

To create a SAVEPOINT, we need to START a TRANSACTION and simply call

SAVEPOINT <savepoint_name>;

This will be basically a “check point” for you to reference and rollback to (basically revert any changes done) if you need to.

Now we can go ahead and DELETE all the bad input data using the DELETE instruction for all the rows that violate these conditions:

1. Names of people or places with NULL values
2. Salary < 0
3. Company not in {ACME Corporation, Bank of America, Chase, Capital One, UniCredit}

Once we have deleted all the bad tuples, we can go ahead and COMMIT the changes which will release our savepoint and finalize the changes.

If at anytime you need to rollback any mistakes, simply call

ROLLBACK TO SAVEPOINT <savepoint_name>;

Part 2 - Adding additional constraints
Goal: Add these constraints back into the database table

Now we need to add constraints (back) into the tables. This will not work if you have not deleted the bad input values. The constraints are the same as the bad input checks from Part 1.

1. Names of people or places cannot be NULL
2. Salary > 0
3. Company in {ACME Corporation, Bank of America, Chase, Capital One, UniCredit}

Once again it is recommended that we do this code within a savepoint.

You can verify these constraints by trying to insert something that violates these checks.

Part 3 - Adding Triggers
Goal: write a trigger to catch the bad value and update it to a different value

The last thing we can do is also add some triggers to aid in good value inserts. In our hypothetical database, companies can hire contractors. When filling out the forms, contractors may put in a 0 for salary, as they are only paid per job completion.

Because of our constraint, we must put a positive, non-zero value into our database. Your boss has permitted you to create a trigger that will catch this value and save the value of 3000 instead, which is the fixed amount contractors make per job.

Verify that the trigger works by inserting (‘George Washington’, ACME corporation, 0) into the works table.